

DOCUMENTO DE DEFENSA TÉCNICA: DETECTOR DE APARCAMIENTO CON VISIÓN ARTIFICIAL

Autores: Equipo de Desarrollo (Pakoies y Javier)
Fecha: Junio de 2026
Asignatura: Proyecto de Visión Artificial e Inteligencia Artificial
Centro: IES Gran Capitán

1. Resumen Ejecutivo y Propósito del Proyecto

El presente proyecto resuelve un problema clásico de la domótica y las ciudades inteligentes (Smart Cities): la gestión eficiente y la monitorización en tiempo real de espacios de aparcamiento. Utilizando cámaras de videovigilancia fijas (o flujos de vídeo simulados en este caso), el sistema automatiza por completo la detección de plazas libres u ocupadas sin necesidad de instalar costosos sensores físicos (sensores magnéticos o de ultrasonidos) en el suelo de cada plaza.

La arquitectura de software diseñada destaca por su enfoque modular, dividiéndose de forma estricta en tres etapas independientes:

- Calibración Geométrica:** Definición previa de las Regiones de Interés (ROI) que representan cada plaza.
- Inferencia e Inteligencia Artificial:** Procesamiento de imágenes mediante la red neuronal convolucional YOLOv8 para la localización de vehículos.
- Interfaz de Usuario (Bot de Telegram):** Exposición asíncrona de los resultados mediante una API conversacional.

2. Arquitectura de Software y Pipeline de Datos

La modularidad del sistema es uno de sus puntos más fuertes. El flujo de datos sigue una secuencia lineal bien definida que garantiza que cada componente realice una única tarea (Principio de Responsabilidad Única):

Componente / Archivo	Entradas primarias	Procesamiento core	Salidas / Artefactos
marcar_plazas.py	Imagen estática de referencia (imgs/1.png)	Captura de eventos de ratón en OpenCV (clics de usuario) para trazar polígonos alrededor de cada plaza libre.	Fichero de configuración espacial imgs/spots.json.

Componente / Archivo	Entradas primarias	Procesamiento core	Salidas / Artefactos
detector_yolo.py	Imagen o frame actual + imgs/spots.json	Inferencia de YOLOv8n para extraer bounding boxes. Algoritmo de intersección de geometrías (IoU / solapamiento) entre cajas y ROIs.	Fichero de estado numérico imgs/estado_actual.json e imagen anotada con recuadros.
bot_telegram.py	Petición del usuario (/estado)	Orquestación asíncrona. Invoca el motor de detección, formatea el texto informativo y prepara los buffers de imagen.	Mensaje de Telegram con estadísticas y la imagen procesada con colores verde/rojo.

3. Análisis Detallado de los Módulos de Código

3.1. Módulo de Calibración: marcar_plazas.py

Este script se ejecuta **una sola vez** o únicamente cuando la cámara cambia de posición física. Su propósito es solventar una limitación nativa de las redes neuronales: YOLO detecta objetos, pero no detecta el "vacío" o la ausencia de ellos. Mediante OpenCV, este script permite mapear el entorno real a coordenadas de píxeles.

- **Uso de Callbacks de Ratón:** Mediante `cv2.setMouseCallback`, se interceptan de forma asíncrona los eventos del sistema operativo. El clic izquierdo añade un vértice a la lista, y el doble clic cierra el polígono calculando la geometría interna.
- **Persistencia de Datos:** Las coordenadas de los polígonos no se pierden al cerrar el programa; se estructuran y serializan en formato JSON dentro de `spots.json`, garantizando que el detector pueda consumirlas directamente.

3.2. Módulo de Inferencia y Lógica Espacial: detector_yolo.py

Es el núcleo matemático y de IA del proyecto. Este componente combina el aprendizaje profundo (Deep Learning) con la geometría computacional clásica.

- **YOLOv8n (Nano):** Se seleccionó la variante Nano del modelo debido a sus extraordinarias métricas de rendimiento. Con un tamaño de archivo mínimo y una latencia de inferencia de apenas unos milisegundos por frame, es ideal para ejecutarse en hardware embebido o servidores con recursos limitados, ofreciendo una tasa de FPS muy fluida.
- **Algoritmo de Solapamiento e Intersección:** Para solventar el problema de los vehículos detectados fuera de las instalaciones (calle, carreteras adyacentes), el script no analiza toda la imagen a ciegas. El programa evalúa si la caja delimitadora (Bounding Box) devuelta por YOLO interseca significativamente con los polígonos definidos en el

archivo JSON. Si hay solapamiento con una zona ROI específica, esa plaza cambia su estado automáticamente a **ocupada** (rojo); si no hay ningún vehículo interfiriendo en la zona geométrica, se computa como **libre** (verde).

3.3. Módulo de Interfaz y API: bot_telegram.py

Representa la capa de presentación y la interfaz que interactúa con los usuarios de forma remota.

- **Programación Asíncrona (Async/Await):** Utiliza la librería moderna python-telegram-bot basada en operaciones no bloqueantes. Esto asegura que el servidor del bot pueda atender múltiples peticiones concurrentes de distintos usuarios simultáneamente sin congelar la ejecución ni retrasar las detecciones.
- **Generación Dinámica de Feedback Visual:** Cuando el usuario introduce el comando /estado, el bot activa el pipeline de detección en tiempo real sobre el último frame disponible. No solo responde con una cadena de texto plana, sino que procesa la imagen en memoria, dibuja las plazas codificadas por colores y envía un archivo multimedia completo, facilitando al usuario final la comprobación visual de qué plazas exactas están disponibles.

4. Respuestas Clave ante el Tribunal / Profesor

Para asegurar la máxima calificación durante la defensa oral, el equipo debe responder con solvencia a las siguientes problemáticas de diseño:

1. **¿Cómo se evita que el sistema cuente coches que pasan por la calle de fuera?**
Respuesta: Gracias a la separación de conceptos. YOLO detecta absolutamente todos los vehículos de la imagen (incluida la calle), pero nuestra lógica implementada en detector_yolo.py actúa como un **filtro espacial estricto**. Cualquier coche cuyas coordenadas de caja delimitadora no tengan una intersección matemática con los polígonos del parking declarados en spots.json es descartado inmediatamente de los cálculos.
2. **¿Por qué es mejor este enfoque que entrenar a YOLO para detectar "plazas vacías"?**
Respuesta: Las redes neuronales de detección se entrenan reconociendo patrones visuales concretos y densos (texturas, ruedas, cristales de un coche). Una "plaza vacía" carece de un patrón visual único: puede ser asfalto gris, suelo de cemento, líneas blancas o reflejos del sol, lo que generaría un índice altísimo de falsos positivos. Nuestro enfoque híbrido (OpenCV + YOLO) delega en la IA lo que mejor sabe hacer (detectar objetos físicos/coches) y usa la geometría clásica para deducir el vacío por ausencia de objeto.
3. **¿Qué ocurre con el problema de la oclusión (vehículos tapados por árboles o farolas)?**
Respuesta: La oclusión es un desafío inherente a la visión artificial monocular (una sola

cámara). Lo mitigamos ajustando el umbral de confianza (--conf) y calibrando los polígonos de las plazas considerando la perspectiva de la cámara. Al evaluar el solapamiento completo del bounding box en lugar de únicamente su punto central, aumentamos la tolerancia del sistema ante oclusiones parciales del vehículo.

5. Conclusión y Trabajo Futuro

El sistema propuesto demuestra una excelente sinergia entre las técnicas de visión artificial clásica (OpenCV) y el aprendizaje profundo moderno (YOLO), logrando un producto software de alta fiabilidad, bajo coste y fácilmente escalable.

Como líneas de desarrollo futuro para expandir el sistema, se plantea la integración de un histórico en base de datos SQLite para realizar analíticas predictivas de ocupación horaria, y el despliegue del software mediante contenedores Docker para facilitar su instalación e integración en cualquier infraestructura de cámaras IPs existentes.